Week 5 Report

Composable Strategy Engine, Optimization & MVP2 Dashboard

Course: [Sum26] Practicum Project
Project: BackTestBench: modular backtesting and sandbox platform for trading strategies
Track: Industrial
TA: Gerald Imbugwa
Customer: Alexey Potemkin
Date: 07.07.2026

1. Team Information

- Team name: BackTestBench
- Primary communication channel: https://t.me/Muratich_29
- Marat Diiarov — Team lead, reports and customer communication, frontend.
- Georgii Beliaev — Tech lead, requirements and architecture, DevOps/QA.
- Bogdan Zinchenko — Data engineering and historical data management.
- Samy Badavi — Simulation Engine development.
- Maxim Kharlamov — Analytics and ranking.
- Philip Oghenerukevwe Idiare — Strategy Module.
- Nikolay Taran — Broker Adapter and T-Bank integration.

2. Project and Track Statement

BackTestBench is an industrial-track project for testing trading strategies on historical Russian market data. After Week 4 delivered plugin strategies, Top-N ranking, and a multi-strategy dashboard, Week 5 focused on the customer's primary ask: a flexible, file-driven strategy system with composable triggers, actions, TP/SL exit rules, and parameter optimization — while keeping the dashboard mainly a visualizer.

By July 7, 2026 the repository supports YAML-defined composable strategies (series + rules + actions), grid and random parameter optimization integrated into main.py, an MVP2 dashboard with explicit Run/Stop controls, instrument dropdown (19 MOEX tickers), optimization results table, single-load candle pipeline for optimizer efficiency, and example TwelveData/Bybit broker adapters. The July 6 customer demo confirmed real progress on the engine abstraction; the headline gap for Week 6 is multi-period stability validation and preset → runtime wiring fixes.

3. Week 5 Goal, Scope, and Milestone Status

Planned goal (from June 29 customer review)

Deliver the flexible strategy architecture prioritized in the Week 4 report:

- 1. P0: Trigger/action abstraction, TP/SL as first-class parameters, file/config-driven strategies (UI stays visualizer).
- 2. P1: Discrete-grid parameter optimizer for at least one strategy.
- 3. P1 UX: Explicit Calculate/Run; freeze ranking until submit completes.
- 4. Supporting: Multi-instrument UI selection (configuration path existed; picker deferred).

Milestone status

Priority item	Status
Composable trigger/action strategy engine	Delivered (PR #127)
TP/SL exit rules in YAML rules with priority ordering	Delivered
Parameter optimizer (grid + random sample)	Delivered (PR #128, #130)
Explicit Run/Stop UX	Delivered (PR #130)
Instrument dropdown (single-instrument)	Partial — 19 tickers, not multi-instrument portfolio
Validation workflow (holdout second stage)	Not integrated — analytics library only
Preset → runtime wiring for all strategies	Open — customer-reported bugs, 4 failing tests
Multi-period stability validation	Deferred to Week 6 — customer headline ask after July 6 demo

4. Implementation Summary

4.1 Composable strategy engine (PR #127)

• Full YAML-based rule engine under src/strategy/composable/:
• Series DAG: SMA, RSI, nested indicators (series.py).
• Predicates: cross_above/below, gte/lte, loss_pct/profit_pct, all/any (predicates.py).
• Rules with priority + scope (flat/long): stop-loss (100), take-profit (90), exit signals (10) (rules.py).
• Actions: buy/sell/hold with TP/SL on entry (actions.py).
• Compile pipeline: YAML → CompiledStrategy (compile.py, definition.py).
• ComposableStrategy registered; dashboard auto-discovers config/strategies/*.yaml via strategy_manifest.py.
• Demo strategies: ma_rsi_composable.yaml, sma_cross_demo.yaml, rsi_bounce_demo.yaml.
• Shared presets in config/param_presets.yaml.
• Design document added in PR #107 (docs/strategy_composable_engine_design.md).
• Tests: tests/unit/strategy/composable/*, tests/integration/test_ma_rsi_composable_e2e.py.
• Fixed (July 7, PR #137): stop_loss_pct / take_profit_pct defaults aligned with param_presets.yaml choices; backend suite expanded to 144 passing tests (82% src/ coverage).

4.2 Optimization engine (PR #128, extended in #130)

• RandomSearchExecutionEngine in src/engine/optimization_engine.py.
• Grid mode: exhaustive combinatorial search over YAML choices marked optimizable: true.
• Sample mode: random subset with reproducible seed + n_iterations.
• Constraint validation (fast < slow, rsi_buy_min < rsi_overbought).
• Integrated into main.py → run_strategy_backtest() auto-optimizes composable strategies; best params saved to config/dashboard.json → strategy_overrides.
• Optimization summary (top iterations) exposed in data/runtime-dashboard.json and MVP2 optimization panel.
• Tests: tests/unit/engine/test_optimization_engine.py.

4.3 MVP2 frontend integration (PR #130, #134)

• Major dashboard refactor in frontend/app/page.tsx:
• BacktestControlPanel: explicit Run backtest / Stop buttons (no per-keystroke recalc).
• Instrument dropdown (19 MOEX tickers), timeframe, lookback, capital, optimization mode/iterations/seed.
• AddStrategyPanel: paste/save composable YAML via API.
• Optimization panel per strategy showing top parameter combinations.
• ParamSummary chips (read-only; editing via YAML/config, not inline per-strategy forms).
• Removed POST /api/run-strategy (Week 4 per-strategy immediate rerun route).
• New routes: GET/POST /api/config, POST /api/stop, POST/DELETE /api/strategies.
• Customer transcription from July 6 demo added (transcriptions/06-07-26-customer.txt).

4.4 Composable data loader (PR #136, issues #118, #119)

• LoadedMarketData dataclass: candles + price_series (timestamp/close only).
• load_price_series() for composable engine consumption.
• ensure_candles_loaded(): single broker fetch per bootstrap, reused for all strategies and optimizer iterations.
• has_sufficient_data() window coverage check.
• main.load_market_data() / bootstrap_all() use one DataLoader instance.
• Tests: tests/integration/test_market_data_cache.py.

4.5 Broker adapters — TwelveData & Bybit (PR #135)

• src/broker_adapter/twelvedata.py — REST /time_series, TWELVEDATA_TOKEN.
• src/broker_adapter/bybit.py — V5 kline API, 200-candle pagination.
• examples/List_of_spots_bybit.txt (512 symbols), extended examples/README.md.
• Not wired into dashboard: main.py still uses TBankAdapter only for live fetches.

4.6 Supporting PRs (#105, #106, #107)

• #105 — Documentation audit baseline and Week 4 status report (June 30).
• #106 — Delete obsolete empty src/engine/simulation.py.
• #107 — Bilingual composable engine design document (~697 lines) before implementation in #127.

4.7 CI/CD hardening (PR #137)

• Replaced self-hosted PR smoke workflow with GitHub-hosted ubuntu-latest jobs:
backend-tests, frontend-checks, docker-smoke.
• Docker smoke uses APP_PORT=8080, standalone Next.js server, and HTTP 200 check.
• Added preset, manifest, optimization, and config tests; fixed composable compile defaults.

5. Updated API Surface

Implemented Week 5 routes (Next.js):

• GET /api/dashboard — multi-strategy state, metrics, chart points, ranking, optimization summary.
• GET / POST /api/config — load/save runtime settings and UI schema.
• POST /api/bootstrap — run all composable strategies (main.py bootstrap).
• POST /api/stop — stop running backtest.
• POST /api/strategies — add composable strategy YAML.
• DELETE /api/strategies/{id} — remove strategy.
• POST /api/refresh-ranking — rebuild Top-N from completed runs.

Removed since Week 4: POST /api/run-strategy (per-strategy immediate rerun).

Planned FastAPI contract in docs/api_description.md remains target architecture, not the live integration path.

6. Industrial Track Contribution

Week 5 delivered the customer's primary product direction from the June 29 review:

• Flexibility: strategies defined as YAML rules (series + predicates + actions), not new Python classes per idea.
• Exit logic ownership: TP/SL as first-class rule parameters with priority ordering (stop-loss 100, take-profit 90).
• Optimization: grid and random search over realistic parameter ranges (~8000 combos demonstrated on July 6).
• UI as visualizer: dashboard shows metrics, charts, ranking, optimization table — parameter editing via YAML and config files, not rigid per-strategy form expansion.
• Data efficiency: single candle load per bootstrap reused across strategies and optimizer iterations.

The July 6 customer review validated engine progress while reframing the final week from "best params on one month" to "prove those params are stable across historical slices."

7. Screenshots / Demo Evidence

Screenshots captured from the live MVP2 dashboard on July 7, 2026 (npm run dev, port 3010).
Week 4-era screenshots (inline per-strategy parameter forms, no Run/Stop panel) are not used here.

Live demo (VM): http://10.93.27.6

MVP2 dashboard — Run/Stop, optimization settings, Add strategy entry point

TRADING ANALYTICS

BackTestBench

Instrument: GAZP | TF: 1d | Source: database

FIND STRATEGY

Name or ID, e.g. MA Crossover [Go to]

Enter a name or ID and press Enter

BACKTEST CONTROL

Run settings

[Run backtest] [Stop]

INSTRUMENT	TIMEFRAME	LOOKBACK DAYS	INITIAL CAPITAL (RUB)
GAZP	1d	900	10000000
MOEX TQBR shares supported by T-Bank adapter	Min resolution 1m. Max lookback for 1d: 2400 days	History window loaded for the backtest	Starting portfolio cash for each strategy

OPTIMIZATION

GRID
Full grid search - evaluates every valid parameter combination from YAML choices. Guarantees the best result on the current dataset; can take several minutes.

SAMPLE
Random sample - evaluates N random combinations (see iterations + seed). Faster, but does not guarantee the global optimum.

OPTIMIZATION ITERATIONS	OPTIMIZATION SEED
16	42
Number of random combinations to evaluate	Fixes the random sample for reproducible runs

STRATEGIES

Registered from config/strategies/

[Add strategy]

#1 SMA Cross Demo
`sma_cross_demo`

Delete | READY | 11 838 267 ₽

ORDER SIZE	FAST	SLOW	STOP LOSS PCT	TAKE PROFIT PCT
1	5	20	1	0.5

PARAMETER SEARCH

Random sample: 16 of 256 combinations (seed 42)

#	P&L	SHARPE	PARAMS
1	1 838 267 ₽	0,92	fast=5, slow=20, stop loss pct=1, take profit pct=0.5
2	1 527 613 ₽	0,74	fast=5, slow=20, stop loss pct=1, take profit pct=1
3	1 489 105 ₽	0,79	fast=5, slow=30, stop loss pct=0.3, take profit pct=0.5
4	1 166 259 ₽	1,27	fast=5, slow=100, stop loss pct=0.5, take profit pct=0.5
5	940 272 ₽	0,91	fast=5, slow=100, stop loss pct=0.7, take profit pct=1

P&L	SHARPE	DRAWDOWN	WIN RATE	DEPOSIT	CAPITAL
1 838 267 ₽	0,92	4,7 %	73,7 %	13 514 151 ₽ Bank baseline	10 000 000 ₽

118.4 (+18.4%) vs 56.9 (-43.1%) α +61.4 pp

Strategy index | Buy & hold | Baseline 100 | BUY | SELL

#2 MA Crossover + RSI (composable)
`ma_rsi_composable`

Delete | READY | 11 692 927 ₽

ORDER SIZE	FAST	SLOW	RSI PERIOD	RSI BUY MIN	RSI OVERBOUGHT	STOP LOSS PCT	TAKE PROFIT PCT
1	5	30	14	60	70	0.7	1

PARAMETER SEARCH

Random sample: 15 of 8640 combinations (seed 42)

#	P&L	SHARPE	PARAMS
1	1 692 927 ₽	1,1	fast=5, slow=30, rsi period=14, rsi buy min=60, rsi overbought=70, stop loss pct=0.7, take profit pct=1
2	1 557 725 ₽	0,72	fast=5, slow=30, rsi period=20, rsi buy min=40, rsi overbought=65, stop loss pct=0.3, take profit pct=1.5
3	1 290 484 ₽	0,75	fast=5, slow=30, rsi period=20, rsi buy min=50, rsi overbought=80, stop loss pct=0.7, take profit pct=0.5
4	1 290 484 ₽	0,75	fast=5, slow=30, rsi period=20, rsi buy min=50, rsi overbought=65, stop loss pct=0.7, take profit pct=0.5
5	450 829 ₽	0,37	fast=50, slow=100, rsi period=20, rsi buy min=50, rsi overbought=70, stop loss pct=0.5, take profit pct=2

P&L	SHARPE	DRAWDOWN	WIN RATE	DEPOSIT	CAPITAL
1 692 927 ₽	1,1	0,8 %	85,7 %	13 514 151 ₽	10 000 000 ₽

MVP2 BacktestBench: Backtest control panel with Run backtest / Stop, instrument dropdown, grid/sample optimization, and Add strategy button above strategy cards

Add strategy — paste composable YAML (no file upload; Save writes to config/strategies/)

TRADING ANALYTICS

BackTestBench

Instrument: GAZP | TF: 1d | Source: database

FIND STRATEGY

Name or ID, e.g. MA Crossover [Go to]

Enter a name or ID and press Enter

BACKTEST CONTROL

Run settings

[Run backtest] [Stop]

INSTRUMENT	TIMEFRAME	LOOKBACK DAYS	INITIAL CAPITAL (RUB)
GAZP	1d	900	10000000
MOEX TQBR shares supported by T-Bank adapter	Min resolution 1m. Max lookback for 1d: 2400 days	History window loaded for the backtest	Starting portfolio cash for each strategy

OPTIMIZATION

GRID

Full grid search - evaluates every valid parameter combination from YAML choices. Guarantees the best result on the current dataset; can take several minutes.

SAMPLE

Random sample - evaluates N random combinations (see iterations + seed). Faster, but does not guarantee the global optimum.

OPTIMIZATION ITERATIONS	OPTIMIZATION SEED
16	42
Number of random combinations to evaluate	Fixes the random sample for reproducible runs

STRATEGIES

Registered from config/strategies/

[Close editor]

STRATEGY YAML

```
# Composable strategy YAML (saved to config/strategies/{name}.yaml)
name: my_strategy
title: My Strategy

params:
  order_size: { type: float, default: 1, choices: [1, 2, 3], optimizable: false }

series:
  sma_fast: { fn: sma, source: price, period: 10 }
  sma_slow: { fn: sma, source: price, period: 30 }

rules:
  - id: entry
    scope: flat
    priority: 10
    when: { cross_above: [sma_fast, sma_slow] }
    then: { action: buy, size: "${order_size}" }

  - id: exit
```

Must include name, params, series, and rules. File is saved as config/strategies/{name}.yaml and picked up on the next dashboard refresh.

[Save strategy] [Cancel]

#1 SMA Cross Demo

sma_cross_demo

[Delete] READY **11 838 267 ₽**

ORDER SIZE	FAST	SLOW	STOP LOSS PCT	TAKE PROFIT PCT
1	5	20	1	0.5

PARAMETER SEARCH

Random sample: 16 of 256 combinations (seed 42)

#	P&L	SHARPE	PARAMS
1	1 838 267 ₽	0,92	fast=5, slow=20, stop loss pct=1, take profit pct=0.5
2	1 527 613 ₽	0,74	fast=5, slow=20, stop loss pct=1, take profit pct=1
3	1 489 105 ₽	0,79	fast=5, slow=30, stop loss pct=0.3, take profit pct=0.5
4	1 166 259 ₽	1,27	fast=5, slow=100, stop loss pct=0.5, take profit pct=0.5
5	940 272 ₽	0,91	fast=5, slow=100, stop loss pct=0.7, take profit pct=1

P&L	SHARPE	DRAWDOWN	WIN RATE	DEPOSIT	CAPITAL
1 838 267 ₽	0,92	4,7 %	73,7 %	13 514 151 ₽ Bank baseline	10 000 000 ₽

118.4 (+18.4%) vs 56.9 (-43.1%) α +61.4 pp

Strategy index Buy & hold Baseline 100 BUY SELL

MA Crossover + RSI (composable)

Add strategy editor open: Strategy YAML textarea with template, Save strategy / Cancel, hint that file is saved as config/strategies/{name}.yaml

Backend test evidence

```
$ pytest tests --cov=src --cov-report=term -q

============================= test session starts ==============================
platform darwin -- Python 3.12.2, pytest-8.3.4, pluggy-1.6.0
rootdir: /Users/georgijbelyaev/ai_video/BackTestBench
configfile: pyproject.toml
plugins: locust-2.43.3, cov-7.1.0, Faker-38.2.0, anyio-3.7.1, asyncio-0.25.0
asyncio: mode=Mode.STRICT, asyncio_default_fixture_loop_scope=None
collected 144 items

tests/integration/test_get_candles.py .                                  [  0%]
tests/integration/test_ma_rsi_composable_e2e.py ..                       [  2%]
tests/integration/test_market_data_cache.py ..                           [  3%]
tests/integration/test_strategy_engine_integration.py ...                [  5%]
tests/unit/analytics/test_analytics.py ..............                    [ 15%]
tests/unit/data_loader/test_loader.py .....                              [ 18%]
tests/unit/data_loader/test_validator.py ...                             [ 20%]
tests/unit/engine/test_engine.py ..                                      [ 22%]
tests/unit/engine/test_optimization_engine.py ..........                 [ 29%]
tests/unit/strategy/composable/test_compile.py ................          [ 40%]
tests/unit/strategy/composable/test_predicates_rules_actions.py .......  [ 45%]
tests/unit/strategy/composable/test_presets.py .........                 [ 52%]
tests/unit/strategy/composable/test_series.py .......                    [ 56%]
tests/unit/strategy/test_config_interface.py ........                    [ 62%]
tests/unit/strategy/test_loader.py ...                                   [ 64%]
tests/unit/strategy/test_ma_rsi.py .....                                 [ 68%]
tests/unit/strategy/test_rsi.py .....                                    [ 71%]
tests/unit/strategy/test_strategy.py ....................                [ 85%]
tests/unit/test_backtest_config.py ..........                            [ 92%]
tests/unit/test_strategy_manifest.py ...........                         [100%]

================================ tests coverage ================================
_______________ coverage: platform darwin, python 3.12.2-final-0 _______________

Name                                          Stmts   Miss  Cover
-----------------------------------------------------------------
src/__init__.py                                   0      0   100%
src/analytics/__init__.py                         4      0   100%
src/analytics/metrics.py                        108     16    85%
src/analytics/ranking.py                         69      4    94%
src/analytics/validation.py                      51      0   100%
src/api/__init__.py                               0      0   100%
src/api/app.py                                    0      0   100%
src/api/routers/__init__.py                       0      0   100%
src/api/routers/backtest.py                       0      0   100%
src/api/routers/data.py                           0      0   100%
src/api/routers/strategies.py                     0      0   100%
```

pytest summary — 144 passed, 82% src coverage, July 7, 2026

CI workflow (GitHub-hosted PR verification)

```
.github/workflows/ci.yml · pull_request → main

name: PR Build & Run

on:
  pull_request:
    branches: [main]

jobs:
  backend-tests:
    runs-on: ubuntu-latest
    if: github.event.action != 'closed'

    steps:
      - name: Checkout PR
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: requirements.txt

      - name: Install Python dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install -r requirements.txt

      - name: Run backend tests
        run: pytest tests -q

  frontend-checks:
    runs-on: ubuntu-latest
    if: github.event.action != 'closed'

    steps:
      - name: Checkout PR
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm
          cache-dependency-path: frontend/package-lock.json

      - name: Install and build frontend
        working-directory: frontend
        run: |
          npm ci
          npm run build

      - name: Lint frontend (non-blocking until page.tsx refactor)
        working-directory: frontend
        continue-on-error: true
```

GitHub Actions ci.yml — backend-tests, frontend-checks, docker-smoke on ubuntu-latest

8. Customer Review and Feedback

Review date

Composable engine and optimizer demonstrated to the customer on 06.07.2026 (transcriptions/06-07-26-customer.txt).

Positive confirmation

• Rules-based strategies work end-to-end: triggers, actions, sizing, TP/SL from rules.

• Strategies live in config files; new definitions parse without rewriting the engine.
• Parameter model accepted: defaults vs optimizable ranges, nested series (RSI → SMA).
• Grid and random optimization demonstrated (~8000 combos, seed/batch sampling).

Issues raised (Week 6 priorities)

P0 — Must have:

• 1. Fix param_presets.yaml → runtime wiring; stop-loss changes (5% → 1.5%) did not always update results.
• 2. Clarify percent semantics (5 vs 0.05; leverage interaction).
• 3. Multi-period stability validation — single-month P&L insufficient; need walk-forward on prior months.
• 4. Stability column/score in optimization ranking table.
• 5. Percentage-based metrics (% return, drawdown %, capital usage %) alongside absolute P&L.

P1 — Should have:

• 6. Simplify optimizer UX ("N random trials" vs seed/batch exposure).
• 7. Chart: dashed line when flat vs solid when in position.
• 8. Fix stop/run button glitches and stale dashboard polling.
• 9. Risk/reward ratio in metrics; do not rank by P&L alone.

P2 — Stretch / post-course:

• 10. "Investigate" drill-down with trade history and YoY comparison.
• 11. Short strategies, higher-TF trend filters, market regime switching.
• 12. Trading bot parity with backtest rules.

Customer direction for Week 6

Move from "find best params for one period" to "show how stable those params are across time." Fix param wiring first; stability validation is the headline feature for the final demo.

9. Updated Backlog

Completed in Week 5

• Composable trigger/action strategy engine with TP/SL (#127).
• Parameter optimizer — grid + random search (#128, #130).
• MVP2 dashboard with Run/Stop, instrument dropdown, optimization panel (#130, #134).
• Composable data loader with single-fetch reuse (#136, #118, #119).
• TwelveData and Bybit example adapters (#135).
• Composable engine design document (#107).
• Week 4 docs audit baseline (#105).
• CI/CD hardening — GitHub-hosted three-job workflow (#137).

Next priority (Week 6)

• Multi-period stability validation (P0): same params on 2–3 prior months; stability score in ranking.
• Percentage metrics (P0/P1): return %, drawdown %, risk/reward alongside P&L.
• UX fixes (P1): chart flat/in-position distinction, optimizer simplification, stop/run reliability.
• Validation workflow: second-stage holdout run using src/analytics/validation.py.
• Full data validation pipeline (#26 remaining: duplicates, missing values beyond OHLC checks).

Deferred

• Multi-broker integration into dashboard (adapters exist as examples).
• Multi-instrument portfolio UI.
• Scheduler, notifications, trading bot, relational run persistence, FastAPI service.

10. Required Links

Repository and planning:

- Repository: https://github.com/BackTest-bench-team/BackTestBench
- Kanban board: https://github.com/orgs/BackTest-bench-team/projects/1
- Issues: https://github.com/BackTest-bench-team/BackTestBench/issues
- Live demo (VM): http://10.93.27.6

Pull requests merged this week:

- #105 — Documentation audit and Week 4 status report
- #106 — Delete obsolete simulation file
- #107 — Composable engine design document
- #127 — Composable strategy engine (series, rules, actions, YAML compile)
- #128 — Random search optimization engine
- #130 — MVP2 frontend, main.py orchestration, config/stop/strategies APIs
- #134 — Customer transcription (July 6 demo)
- #135 — TwelveData and Bybit adapters, broker examples
- #136 — Composable data loader, single-fetch cache (LoadedMarketData)
- #137 — GitHub-hosted CI, preset/optimization test fixes, 144-test suite

Documentation:

- Composable engine design: docs/strategy_composable_engine_design.md
- Strategy architecture: docs/strategy_module_architecture.md
- API reference: docs/api_description.md
- Customer notes: transcriptions/06-07-26-customer.txt

11. Verification

Commands run on July 7, 2026 (updated after PR #137):

- Backend: 144 passed — pytest tests -q.
- Preset defaults aligned with param_presets.yaml; optimization tests use ISO timestamps.
- Coverage: 82% of src/ — pytest tests --cov=src --cov-report=term.
- Frontend: production build succeeded after npm ci — Next.js 16; routes /, /api/bootstrap, /api/config, /api/dashboard, /api/stop, /api/strategies, /api/refresh-ranking.
- Frontend lint: 5 errors in page.tsx (react-hooks/set-state-in-effect) — non-blocking in CI.
- Docker: docker compose build OK; smoke with APP_PORT=8080 returns HTTP 200; standalone server (node server.js) replaces incompatible npm run start.
- CI: GitHub-hosted ubuntu-latest — backend-tests, frontend-checks, docker-smoke (see screenshot above).

Non-blocking warnings:

- pytest-asyncio default fixture loop scope deprecation warning.
- Next.js workspace-root inference warning (multiple lockfiles).
- npm audit reports moderate dependency advisories.

Conclusion

Week 5 delivered the Flexible Strategy System milestone: composable YAML strategies with TP/SL exit rules, grid/random parameter optimization, an MVP2 Run-centric dashboard, and an efficient single-load data path. The June 29 customer priorities around file-driven strategy design and optimization are largely met in code.

The July 6 customer review confirmed engine progress while setting Week 6 priorities around preset wiring fixes, multi-period stability validation, and percentage-based metrics — shifting the product value from "best params on one window" to "prove stability across time."